In-Vehicle Infotainment (IVI) Systems

Field	Details
Programme	Wipro Automotive project
Submitted by	Debjit Karmakar
Enrollment no.	12023002002079

1. Objective

Design a conceptual architecture of an in-vehicle infotainment (IVI) system that integrates media, navigation and phone projection features, and annotate the component interfaces and data flow between the blocks.

2. Background

An IVI system combines entertainment, navigation, connectivity and vehicle information on the head unit. Modern systems are built on a layered software stack, such as Android Automotive OS, so that applications stay independent of the underlying audio, positioning and vehicle hardware. The design in this report follows that layered approach: applications call framework services, services talk to hardware abstraction layers (HALs), and HALs drive the physical hardware.

3. Approach

- **Data-driven design.** The architecture was described as data in three CSV files: components, interfaces and data-flow scenarios.
- **Notebook.** A Colab notebook loads the datasets, checks that every interface and scenario step refers to a real component, and draws the diagrams directly from the data.
- **Scenario tracing.** Four scenarios were traced hop by hop through the stack to show how an event propagates from input to output.

Note: the latency values used in the scenarios are assumed design-target estimates for illustration. They are not measurements from a real vehicle.

4. Architecture

The system has five layers and four feature columns. Media, navigation and projection each have their own vertical path from application to hardware, and a fourth vehicle column provides the system UI, the Car service and vehicle bus access.

Figure 1 - IVI conceptual architecture (media, navigation, projection)

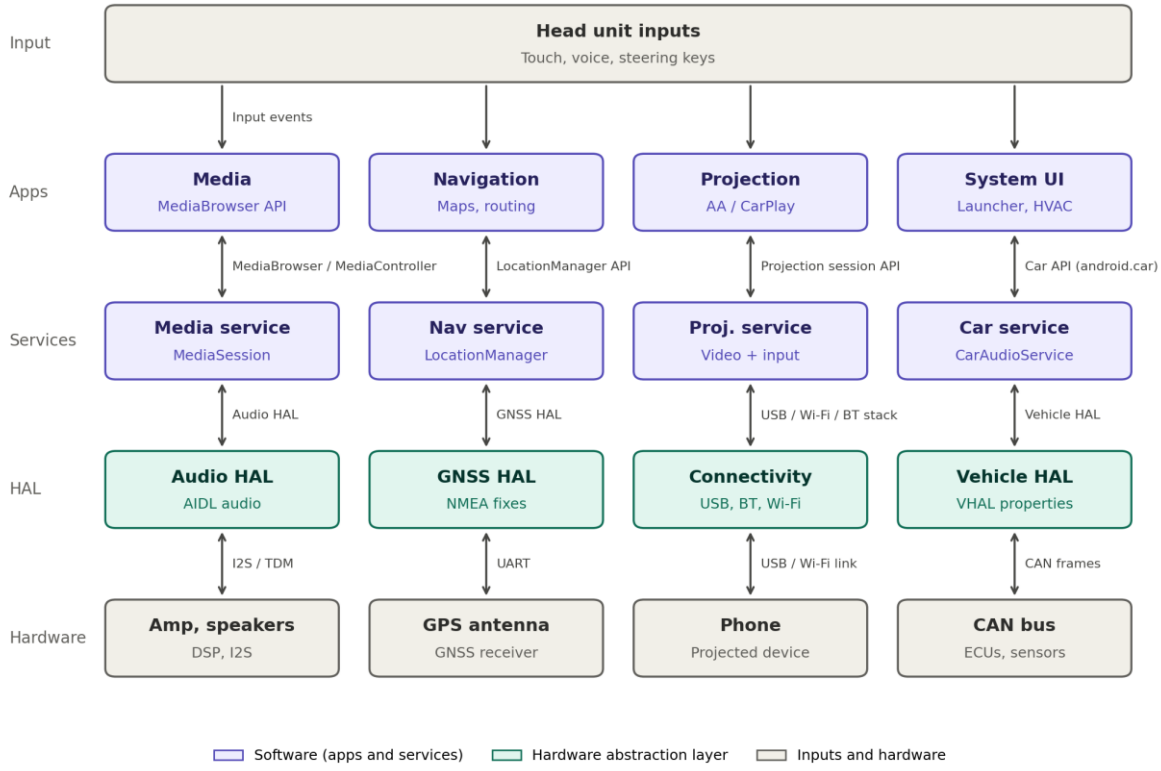


Figure 1: IVI conceptual architecture (generated by the notebook from the datasets)

4.1 Layers

- **Input:** Touchscreen, voice and steering-wheel keys deliver user events to the apps.
- **Apps:** Media, Navigation, Projection (Android Auto / CarPlay) and System UI.
- **Services:** Media service (playback sessions), Nav service (location), Projection service (video and input) and the Car service (audio focus, power, driving restrictions).
- **HAL:** Audio HAL, GNSS HAL, Connectivity (USB, Bluetooth, Wi-Fi) and Vehicle HAL hide hardware differences from the framework.
- **Hardware:** Amplifier and speakers, GPS antenna, the connected phone and the vehicle CAN bus.

4.2 Components

Component	Layer	Technology	Role
Head unit inputs	Input	Android InputManager	Driver interaction channels: capacitive touchscreen, voice assistant microphone array and steering-wheel keys.
Media	Apps	Android Automotive media app	Music, radio and podcast app that browses and plays content.
Navigation	Apps	Android Automotive navigation app	Map rendering, route guidance and turn-by-turn prompts.
Projection	Apps	Projection client	Renders the phone UI on the head unit (Android Auto or Apple CarPlay).
System UI	Apps	Car launcher / SystemUI	Launcher, status bar, climate and vehicle settings screens.
Media service	Services	MediaSessionService	Owns playback sessions, queue and transport controls.

Component	Layer	Technology	Role
Nav service	Services	LocationManagerService	Delivers location fixes and manages guidance sessions.
Proj. service	Services	Projection host service	Hosts the projection session: video decode, input forwarding, audio routing.
Car service	Services	CarService	Central automotive service: audio focus and zones, power, driving-state restrictions.
Audio HAL	HAL	Audio HAL (AIDL)	Abstracts the audio DSP and amplifier for streams, volume and routing.
GNSS HAL	HAL	GNSS HAL (AIDL)	Abstracts the GNSS receiver and reports position fixes.
Connectivity	HAL	USB / BT / Wi-Fi stack	USB accessory, Bluetooth and Wi-Fi transports to the phone.
Vehicle HAL	HAL	Vehicle HAL (AIDL)	Exposes vehicle properties (speed, gear, ignition) to Android.
Amp, speakers	Hardware	I2S / TDM	Audio DSP, power amplifier and speaker system.
GPS antenna	Hardware	UART / NMEA	Active GNSS antenna and receiver chip.
Phone	Hardware	USB AOA / Wi-Fi	Driver's smartphone connected by cable or wireless link.
CAN bus	Hardware	CAN	Vehicle network connecting body, powertrain and safety ECUs.

5. Component interfaces

Each link between blocks is annotated with the interface name, the protocol and the data it carries. Rows I01 to I16 are the vertical links in Figure 1. Rows I17 to I19 are the audio-focus links shown in Figure 2.

ID	From	To	Interface	Protocol	Data carried
I01	Head unit inputs	Media	Input events	InputManager	Touch, voice and steering-key events
I02	Head unit inputs	Navigation	Input events	InputManager	Touch, voice and steering-key events
I03	Head unit inputs	Projection	Input events	InputManager	Touch, voice and steering-key events
I04	Head unit inputs	System UI	Input events	InputManager	Touch, voice and steering-key events
I05	Media	Media service	MediaBrowser / MediaController	Binder IPC	Browse, play, pause, next, metadata
I06	Navigation	Nav service	LocationManager API	Binder IPC	Location updates, route progress
I07	Projection	Proj. service	Projection session API	Binder IPC / sockets	Video frames, touch events
I08	System UI	Car service	Car API (android.car)	Binder IPC	Vehicle state, settings
I09	Media service	Audio HAL	Audio HAL	AIDL	PCM streams, volume, routing
I10	Nav service	GNSS HAL	GNSS HAL	AIDL	Position fixes, NMEA sentences
I11	Proj. service	Connectivity	USB / Wi-Fi / BT stack	USB AOA, Wi-Fi, BT	Video, audio and input packets
I12	Car service	Vehicle HAL	Vehicle HAL	AIDL	Vehicle property get/set/subscribe
I13	Audio HAL	Amp, speakers	I2S / TDM	I2S	PCM audio samples
I14	GNSS HAL	GPS antenna	UART	UART / NMEA	Raw GNSS data
I15	Connectivity	Phone	USB / Wi-Fi link	USB / Wi-Fi	Projection stream

ID	From	To	Interface	Protocol	Data carried
I16	Vehicle HAL	CAN bus	CAN frames	CAN	Speed, gear, ignition frames
I17	Media service	Car service	Audio focus	Binder IPC	Focus request, loss and abandon callbacks
I18	Nav service	Car service	Audio focus	Binder IPC	Guidance prompt focus request
I19	Proj. service	Car service	Audio focus	Binder IPC	Call and voice focus request

6. Data flow

6.1 Audio-focus arbitration

Media playback, navigation prompts and phone calls share the same speakers. The three services therefore request audio focus from the Car service, which decides who may play. This is why an incoming call pauses media, while a navigation prompt only lowers (ducks) its volume.

Figure 2 - Audio-focus arbitration through the Car service

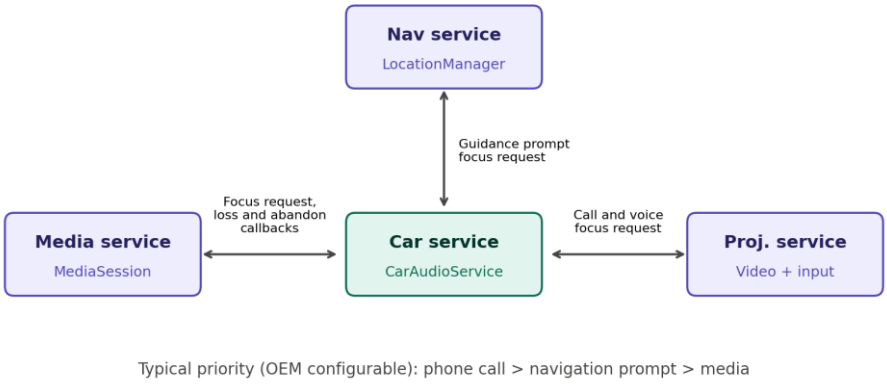


Figure 2: Audio-focus arbitration through the Car service

6.2 Scenario traces

Four scenarios were traced through the architecture. Each row is one hop with its assumed latency.

S1: Play a song from the touchscreen (total 54 ms, assumed)

Step	From	To	Message	ms
1	Head unit inputs	Media	Touch event delivered to Media app	8
2	Media	Media service	MediaController.play() over Binder	5
3	Media service	Car service	requestAudioFocus(GAIN)	6
4	Media service	Audio HAL	Open output stream and write PCM	15
5	Audio HAL	Amp, speakers	PCM over I2S to amplifier	20

S2: Navigation prompt ducks media (total 72 ms, assumed)

Step	From	To	Message	ms
1	GPS antenna	GNSS HAL	GNSS fix received	20
2	GNSS HAL	Nav service	Location update	5
3	Nav service	Navigation	Route progress: turn ahead	5

Step	From	To	Message	ms
4	Nav service	Car service	requestAudioFocus(TRANSIENT_MAY_DUCK)	6
5	Car service	Media service	onAudioFocusChange(LOSS_TRANSIENT_CAN_DUCK)	6
6	Media service	Audio HAL	Duck media volume, route prompt	10
7	Audio HAL	Amp, speakers	Mixed PCM to amplifier	20

S3: Incoming call during projection (total 89 ms, assumed)

Step	From	To	Message	ms
1	Phone	Connectivity	Call state change over projection link	15
2	Connectivity	Proj. service	Event to projection host	5
3	Proj. service	Car service	requestAudioFocus(GAIN_TRANSIENT)	6
4	Car service	Media service	onAudioFocusChange(LOSS_TRANSIENT)	6
5	Media service	Audio HAL	Pause media, flush stream	12
6	Proj. service	Projection	Render call screen	25
7	Audio HAL	Amp, speakers	Route call audio to speakers	20

S4: Reverse gear shows rear camera (total 25 ms, assumed)

Step	From	To	Message	ms
1	CAN bus	Vehicle HAL	Gear = REVERSE frame	10
2	Vehicle HAL	Car service	VHAL property change event	5
3	Car service	System UI	Switch to rear-camera view	10

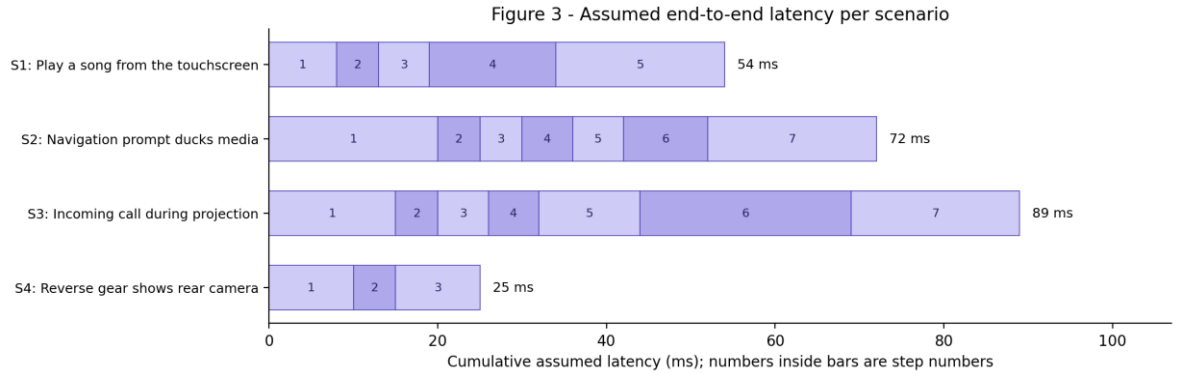


Figure 3: Assumed end-to-end latency per scenario

7. Notebook output

Running the Colab notebook produces the following validation and summary output.

```
components: 17 rows | interfaces: 19 rows | scenario steps: 22 rows
Interfaces with unknown endpoints    : 0
Scenario steps with unknown endpoints: 0

IVI architecture summary
Layers                 : 5
Feature columns        : 4 (media, navigation, projection, vehicle)
Components             : 17
Interfaces             : 19 (16 vertical, 3 audio-focus)
Scenarios / total steps: 4 / 22
Longest scenario       : S3 - Incoming call during projection (89 ms assumed)
```

8. Conclusion

The layered architecture keeps media, navigation, projection and vehicle functions independent, so each can be developed and tested separately. The Car service coordinates audio focus across all features, which allows phone calls and navigation prompts to interrupt media in a controlled way. Because the diagrams are generated from CSV files, a new feature only needs new rows in the datasets.

Possible extensions

- Add a rear-camera video pipeline and a driver-monitoring feature to the vehicle column.
- Replace the assumed latencies with real values measured from Android logs on an emulator or a Raspberry Pi setup (Topics 2 and 3).
- Add security zones, for example separating the infotainment domain from safety-critical ECUs behind a gateway.